

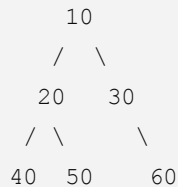
Data Structures Notes: Tree Operations on Binary Tree

1. Introduction to a Binary Tree

A **binary tree** is a non-linear data structure in which each node has **at most two children**:

- **Left child**
- **Right child**

Example:



Here:

- 10 → Root node
- 20 , 30 → Children of 10
- 40 , 50 → Children of 20
- 60 → Right child of 30

2. Basic Binary Tree Node

A binary tree node generally contains three fields:

```
+-----+-----+-----+
| Left  | Data  | Right  |
+-----+-----+-----+
```

In C:

```
C

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```

3. Major Operations on a Binary Tree

Common binary tree operations are:

1. Create a node
 2. Insert a node
 3. Delete a node
 4. Search for a node
 5. Tree traversal
 - Preorder
 - Inorder
 - Postorder
 - Level order
 6. Find height/depth
 7. Count nodes
 8. Count leaf nodes
 9. Count internal nodes
 10. Find minimum and maximum values
-

4. Creating a Node

A new node is created dynamically using `malloc()` .

C

```
struct Node* createNode(int value)
{
    struct Node* newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}
```

For example:

```
createNode(10)
```

```
    10
   /  \
  NULL NULL
```

5. Insertion in a Binary Tree

Insertion depends on the type of binary tree.

For a **general binary tree**, one common approach is **level-order insertion**.

Example:

Initial tree:

```
    10
   /  \
  20   30
```

Insert 40 :

```
    10
   /  \
  20   30
 /
40
```

Insert 50 :

```
    10
   /  \
  20   30
 /  \
40  50
```

Level-order insertion

We use a **queue** to find the first available position.

Algorithm:

1. Create a new node.
 2. If tree is empty, make it the root.
 3. Otherwise, insert the root into a queue.
 4. Remove a node from the queue.
 5. If its left child is NULL, insert the new node there.
 6. Otherwise, insert the left child into the queue.
 7. If its right child is NULL, insert the new node there.
 8. Otherwise, insert the right child into the queue.
 9. Repeat until insertion is completed.
-

6. Searching in a Binary Tree

Searching means finding whether a particular value exists in the tree.

Example:



Search for 50 :

10 → 20 → 40 → 50

A general binary tree can be searched using **DFS** or **BFS**.

Recursive search

C

```
int search(struct Node *root, int key)
{
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    if (search(root->left, key))
        return 1;

    return search(root->right, key);
}
```

7. Tree Traversal

Traversal means visiting every node of a tree exactly once.

The major traversal methods are:

Traversal	Order
Preorder	Root → Left → Right

Traversal	Order
Inorder	Left → Root → Right
Postorder	Left → Right → Root
Level Order	Level by level

Consider:

```
      10
     /  \
    20   30
   / \  / \
  40 50 60 70
```

7.1 Preorder Traversal

Order:

Root → Left → Right

For the above tree:

10 20 40 50 30 60 70

Algorithm

PREORDER(root)

1. Visit root
2. Traverse left subtree
3. Traverse right subtree

C function

C

```
void preorder(struct Node *root)
{
    if (root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
    }
}
```

```
        preorder(root->right);
    }
}
```

8. Inorder Traversal

Order:

Left → Root → Right

For the above tree:

40 20 50 10 60 30 70

Algorithm

INORDER(root)

1. Traverse left subtree
2. Visit root
3. Traverse right subtree

C function

C

```
void inorder(struct Node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

Important Point

For a **Binary Search Tree (BST)**, inorder traversal produces values in **sorted order**.

9. Postorder Traversal

Order:

Left → Right → Root

For the above tree:

40 50 20 60 70 30 10

Algorithm

POSTORDER(root)

1. Traverse left subtree
2. Traverse right subtree
3. Visit root

C function

C

```
void postorder(struct Node *root)
{
    if (root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

Application

Postorder traversal is useful for:

- Deleting a tree
- Evaluating expression trees
- Processing child nodes before parent nodes

10. Level Order Traversal

Level-order traversal visits nodes **level by level from left to right**.

Example:

```
    10
   /  \
  20   30
```

```
  / \   / \
40 50 60 70
```

Output:

```
10 20 30 40 50 60 70
```

It uses a **queue**.

Algorithm

1. Insert root into queue.
2. Repeat while queue is not empty:
 - a. Remove a node.
 - b. Visit the node.
 - c. Insert its left child into queue.
 - d. Insert its right child into queue.

11. Deletion in a Binary Tree

Deletion in a general binary tree is different from deletion in a BST.

A common method is:

1. Find the node to delete.
2. Find the deepest/rightmost node.
3. Copy the deepest node's value into the node to be deleted.
4. Delete the deepest node.

Example:

```
    10
   /  \
  20   30
 /  \  /
40 50 60
```

Delete **20**.

The deepest node is **60**.

Replace **20** with **60**:

```
    10
   /  \
```

```
    60   30
   /  \
  40   50
```

Then remove the original 60 :

```
    10
   /  \
  60   30
 /  \
40  50
```

12. Finding Height of a Binary Tree

The **height** of a tree is the number of edges on the longest path from the root to a leaf.

Example:

```
    10
   /  \
  20   30
 /  \
40  50
```

Longest path:

10 → 20 → 40

Height:

2

Recursive algorithm

```
height(root)

if root == NULL
    return -1

leftHeight = height(root->left)
rightHeight = height(root->right)

return 1 + maximum(leftHeight, rightHeight)
```

C function

C

```
int height(struct Node *root)
{
    int leftHeight, rightHeight;

    if (root == NULL)
        return -1;

    leftHeight = height(root->left);
    rightHeight = height(root->right);

    return 1 + (leftHeight > rightHeight ?
                leftHeight : rightHeight);
}
```

Some textbooks define height in terms of **number of nodes** rather than edges. Under that convention, the same tree would have height **3**.

13. Counting Nodes

To count all nodes:

C

```
int countNodes(struct Node *root)
{
    if (root == NULL)
        return 0;

    return 1 + countNodes(root->left)
            + countNodes(root->right);
}
```

For:

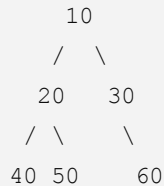
```
    10
   /  \
  20   30
 /  \
40  50
```

Number of nodes:

14. Counting Leaf Nodes

A **leaf node** is a node having no children.

Example:



Leaf nodes:

40, 50, 60

Number of leaf nodes:

3

C function

C

```
int countLeaf(struct Node *root)
{
    if (root == NULL)
        return 0;

    if (root->left == NULL && root->right == NULL)
        return 1;

    return countLeaf(root->left)
        + countLeaf(root->right);
}
```

15. Counting Internal Nodes

An **internal node** is a node having at least one child.

```

      10
     /  \
    20   30
   /  \
  40  50

```

Internal nodes:

10, 20

Therefore:

Internal nodes = 2

C function:

```

C

int countInternal(struct Node *root)
{
    if (root == NULL ||
        (root->left == NULL && root->right == NULL))
        return 0;

    return 1 + countInternal(root->left)
              + countInternal(root->right);
}

```

16. Finding Minimum and Maximum

For a **general binary tree**, we must examine all nodes.

Example:

```

      50
     /  \
    20   80
   / \  /
  10 30 60

```

Minimum:

10

Maximum:

In a **Binary Search Tree**, finding minimum and maximum is more efficient:

Minimum → leftmost node

Maximum → rightmost node

17. Time Complexity of Binary Tree Operations

For a general binary tree containing n nodes:

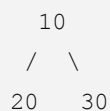
Operation	Time Complexity
Search	$O(n)$
Traversal	$O(n)$
Count nodes	$O(n)$
Count leaf nodes	$O(n)$
Find height	$O(n)$
Find minimum	$O(n)$
Find maximum	$O(n)$
Level-order traversal	$O(n)$

For traversal, every node is visited once.

18. Recursive Nature of Binary Trees

A binary tree is naturally recursive because every node can itself be considered the root of a smaller binary tree.

For example:



The left subtree:

20

and the right subtree:

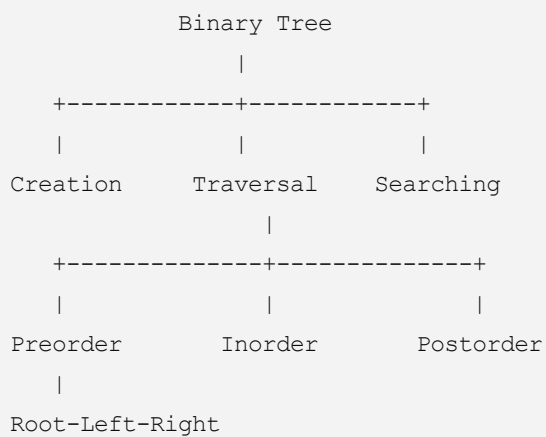
30

are themselves binary trees.

This is why **recursion is commonly used** for:

- Preorder
- Inorder
- Postorder
- Searching
- Counting nodes
- Finding height

19. Summary of Binary Tree Operations



Other Operations:

- Insertion
- Deletion
- Height
- Count Nodes
- Count Leaves
- Min/Max
- Level Order

Important Exam Points

1. A binary tree node has **at most two children**.
2. The two children are called **left child** and **right child**.
3. Preorder = **Root** → **Left** → **Right**.
4. Inorder = **Left** → **Root** → **Right**.
5. Postorder = **Left** → **Right** → **Root**.
6. Level order uses a **queue**.
7. DFS-based traversals commonly use **recursion**.
8. Inorder traversal of a **BST** gives sorted order.
9. A node with no children is called a **leaf node**.
10. The topmost node is called the **root node**.
11. A tree with n nodes has $n - 1$ edges if it is a non-empty tree.
12. Traversing a binary tree takes **$O(n)$** time.